

- RPC**
 - Stubs on either side for ease of use.
 - At-most-one semantics (exactly once on error). Use sequence number to detect duplicate requests. Reject old requests. Accept new requests.
 - Grapevine as a distributed database. Store tables in client and server Server: a) IP address table; b) Export table -> service:dispatcher:uniq-server-id. Protects routines via dispatchers. Unique service id used for restarts after crash -> rejected. c) Call Table: activity:last-seq-num:last-result. Activity => machine:pid.
 - State per connection in server: Export Table, Call Table. One entry per service in export table. One entry per client in Call Table.
 - Remembers the last 1 response. Detects duplicate, old and new calls. Cannot have multiple outstanding requests.
 - Seq number just needs to be monotonically increasing, does not need to be sequential.
 - Server crash: uuid changes, client needs to retry req. Client crash: Server does some useless work (old incarnation). Note: Client needs to make seq number as some function of time so new incarnation will have seq number > call table entry.
 - RPCs cannot be used with UNIX semantics for read/writes read/write with an fd implicitly moves the fd pointer (non-idempotent). Correct approach: use pwrite and pass along the particular offset.

- Failures**
 - Reliability: Not doing the wrong thing. Availability: Doing right thing in specified amount of time. Availability = $(MTBF / (MTBF + MTTR))$.
 - Infant mortality bugs: New bugs slowly discovered. Heisenbug: Non-deterministic, weird bugs. Bohrbugs: Deterministic, removable through testing.
 - Software bugs: Wait for release cycle. Hardware bugs: Roll out ASAP.

- Redundancy**
 - Redundancy != fault tolerance -- All 8 systems studied (Redis, ZooKeeper, Kafka, MongoDB, etc.) suffered data loss, corruption, or unavailability from a single fa...
 - Two fault types: corruptions vs. I/O errors -- Local FS faults are either disk block corruptions (silent bad data) or latent sector errors (I/O error returned). Handle them differently.
 - Undetected local faults -> silent global corruption -- When a node doesn't detect corruption (e.g. Cassandra with checksums off), it silently serves bad data. Distributed layer never gets signal to use healthy replica.
 - Crashing is the most common reaction -- and it backfires -- Fault is persistent so node stays down, redundancy is reduced, cluster risks unavailability without ever using healthy replicas.
 - Crash handling and corruption handling are entangled -- Kafka uses checksums to infer crash data loss, but disk corruption also triggers checksum mismatch. Result: committed data truncated as if a crash artifact.
 - Local behavior has global consequences -- A locally truncated log (Kafka) can allow faulted node to be elected leader, causing cluster-wide data loss. Local recovery must be designed with distributed protocol in mind.

- NFS**
 - All use RPCs.
 - NFSv2 -- NVRAM write buffer; reply cache for non-idempotent (mkdir) ops; getattr overwhelming due to cache coherency checks.
 - NFSv3 -- async-writes (can lose data written to server) -> write verified filled in by both client and server useful for restarts; 64-bit file size; getattr less frequent -> any call returns attr; pre-op and post-op time for write to large files for better cache coherency.
 - 2 concurrent clients writing to the same file can be intermixed on server -- no version gives implicit locking. Even NFSv4 provides locking mechanisms but does not enable by default.
 - NFSv4 -- Compound statements (can interleave, not a transaction, complexity-failure in middle); Open and Close (file locking, consistency, performance); Delegations (performance improvement).
 - NFSv4 can keep a future open() local to client with delegations (read-only/exclusive).
 - How to know if delegation expired -- callback, lease period expiration. Adv: avoid getattr(), keep ops local.
 - Delegations are performance improvements, not correctness -> recovery is optimistic.
 - NFSv4 client crashes -- verifier id changes -- release all locks (lease expires also releases locks).
 - All can keep writes buffered locally until close(). Only NFSv4 can keep buffered locally after close.

- AFS**
 - Whole file caching; Prototype to throw away. Problems: TestAuth, GetFileStat high; CPU load-pathname traversal, process per client-context switch, fork-overhead; load-imb...
 - open-to-close() semantics; callbacks for cache coherency.
 - +Scalability, +Clearer-consistency, -Server state.
 - Lease term: v1-poll on every open => 0; v2-Callbacks => infinite.

- CODA**
 - Pessimistic vs Optimistic replica control. First vs Second class replica. Server replication: Volume Storage Group.
 - Hoarding: hoard-db -- user set priorities, inf priority-ancestor directories. Emulation. Reintegration.

- Leases + Dist. Snapshot**
 - **Why Ordering Matters:** Needed for mutual exclusion, dedup, RSM, snapshots, make, distributed debugging.
 - **Physical Clocks Unreliable:** Clocks drift (~50ms skew in datacenter). Central timeserver = bottleneck. Can't adjust backwards.
 - **Happened-Before (->):** (1) same-process order, (2) send -> receive, (3) transitive. Means 'a' *could* causally affect 'b'.
 - **Concurrent (||):** a || b if a does not happen-before b and b does not happen-before a. Concurrency is **not** transitive.
 - **Lamport Clocks:** Increment on every event (IR1). Send: attach timestamp. Receive: $C_j = \max(C_j, T_m) + 1$ (IR2).
 - **Lamport Guarantee:** a -> b implies $C(a) < C(b)$. Converse is **false** -- can't infer causality from clock values alone.
 - **Total Order:** Break ties with process ID: a => b if $C(a) < C(b)$, or equal clocks and $P_i < P_j$. Non-unique -- concurrent ops can go in any order.
 - **Vector Clocks:** Each process holds vector $[C1..Cn]$; increments own entry locally, takes element-wise max on receive. $C(a) < C(b)$ iff a -> b; incomparable implies a || b. Drawback: size scales with # processes.
 - **Snapshot Goal:** Record global state (all process states + all channel states) without stopping system. Detect stable properties: deadlock, termination, token counts.
 - **Consistent Cut:** A cut is consistent if: whenever event i is in the cut and j -> i, then j is also in the cut. No recorded receive without its corresponding send.
 - **System Assumptions:** Channels are FIFO, error-free, infinite buffers, finite delay. Channel state = messages sent but not yet received.
 - **Chandy-Lamport Initiation:** Any process can start -- records own state, sends a **marker** on all outgoing channels before any further messages.
 - **Marker Receipt (first time):** Record own state; record incoming channel as **empty**; forward marker on all outgoing channels.
 - **Marker Receipt (subsequent):** Record channel state as all messages received on that channel since own snapshot was taken.
 - **Termination:** Snapshot complete when all processes and channels recorded. Requires separate collection step to assemble global picture.
 - **Recorded State S* May Not Be Real:** S* might never have actually occurred. However, S* is reachable from initial state, and final state is reachable from S*. Stable properties valid in S* are still meaningful.
 - **Why S* Is Still Useful:** To reconcile S* with a true logical cut, swap orderings of concurrent pre- and post-snapshot events -- always valid since concurrent op ordering doesn't matter.

Replicated SM

- **Fault-Tolerance:** System continues operating correctly despite failures. Any component can fail -- network, storage, hardware, software.
- **Failure Types:** Fail-stop = nodes go silent. Asynchronous network = messages lost/corrupted/reordered/delayed. Byzantine = malicious nodes sending contradictory/corrupted messages.
- **Measuring Fault-Tolerance:** MTBF/MTTR are client-centric. More useful: t-fault-tolerant -- operates correctly with up to t concurrent failures. Fail-stop needs t+1 replicas; Byzantine needs 2t+1.
- **Correctness = Responsive + Consistent:** Must remain available (timely replies) and consistent (obey semantic contract). Strongest level: **linearizability** -- appears as single atomic server.
- **RSM / SMR Idea:** Model service as deterministic state machine. Replicate correctly-ordered log of commands across replicas. Same state + same log = same outcome.
- **State Machine Requirements:** Must be *deterministic* (no random choices), *single-sequence* (concurrent requests serialized), use *logical time* (no dependence on physical clocks).
- **Why Replicate the Log, Not the State?** Commands are small; state can be huge. Replicas only need to agree on ordered log.
- **Replica Coordination -- Agreement (IC1/IC2):** Every non-faulty replica must receive every command (IC1); if transmitter is non-faulty, all replicas use its value (IC2). Via reliable broadcast.
- **Replica Coordination -- Ordering:** Requests tagged with unique IDs; stable once no lower-ID request can arrive later. Methods: (1) Lamport logical timestamps with FIFO channels; (2) synchronized real-time clocks with known max skew; (3) Paxos-like majority voting.
- **When to Weaken Agreement/Ordering?** Idempotent commands can execute multiple times. Read-only commands can serve from one replica. Commutative commands (EPaxos) don't need strict ordering.
- **Faulty Output Devices:** Replicate outputs -- send from all replicas. External clients handle multiple identical outputs.
- **Faulty Clients:** Fail-stop client won't release locks. Byzantine client sends contradictory inputs. Solutions: replicate client + buffer until enough identical requests; defensive programming with timeouts.
- **Reconfiguration:** Config changes agreed via consensus, scheduled for future log position so all replicas/clients learn new config before it takes effect.
- **Summary:** Replication = ordering + deterministic execution. SMR reduces fault-tolerance to log consensus. Underlies Paxos, Raft, Byzantine replication, transactional serializability.

HA-NFS + REMUS

- **HA-NFS vs Remus:** Both use primary/backup, but HA-NFS replicates at service layer (shared disk) while Remus replicates at VM/infrastructure la...
- **Primary-Backup key insight:** Backup is *passive* -- ordering and determinism handled by primary, which sends state changes rather than raw inputs.
- **Failure detection tradeoff:** Short heartbeat timeout risks false positives; long timeout means longer unavailability. HA-NFS uses 10s + 5s conservatively.
- **When to ACK the client:** ACKing before backup confirms = better latency but risks data loss; waiting for backup ACK = correct but high latency.
- **HA-NFS keys:** Dual-ported shared disks (one writer guarantee), AIXv3 journaling filesystem for crash consistency, NFS v2 stateless server simplifies recovery.
- **Journaling:** Commit = transaction written to log. Crash after commit -> replay journal. Crash before commit -> no update (atomic). Checkpoint periodically flushes to in-place disk locations.
- **HA-NFS takeover:** Backup replays AIX journal exactly like a local crash recovery -- stateless NFS v2 means no in-memory state to reconstruct. Clients just retry timed-out RPCs.
- **Remus approach:** Replicates entire VM state asynchronously; buffers outgoing network output until checkpoint is committed to backup (*speculative execution*).
- **Remus performance costs:** VM overhead, checkpoint pause + dirty page transfer, network buffering latency. Worst for write/memory-intensive workloads; read/network-heavy workloads fare better.
- **Checkpoint interval tradeoff:** Shorter = less data loss, smaller rollback window, but higher overhead. Longer = lower overhead but more exposure on failure.

Serverless

- **Cloud computing core value:** Elastic provisioning, pay-as-you-go, no upfront commitment -- shifts capital to operational expenses and matches capacity to d...
- **XaaS hierarchy:** SaaS (full apps), IaaS (VMs, billed per allocation), FaaS (functions, billed per invocation x time x memory) -- each layer abstracts more from the user.
- **Serverless = FaaS:** Users deploy stateless, bounded-time functions; cloud provider handles all provisioning, scaling (0 to many), and resource management automatically.
- **Virtualization evolution:** Bare metal -> VMs (shared hardware) -> Containers (shared OS) -> Serverless (shared runtime) -- each generation shares more and reduces user responsibility.
- **Internal FaaS execution model:** Load balancer routes to workers; admission controller reuses or creates sandboxes; sandbox evictor frees cold ones under memory pressure; Linux cgroups/CFS enforce limits.
- **Cold start problem:** Starting a new container requires initializing runtime, installing dependencies -- typically 0.2 to a few seconds, a key serverless limitation.
- **Current billing is SLIM:** Static (same memory limit), Linear (CPU tied to memory), Interactive-only (no discount for low-demand time) -- simple for providers but not true pay-for-use.
- **SLIM is profitable but unfair:** Customers pay for reserved-but-unused headroom; small-input invocations overpay relative to actual consumption.
- **Nearly-PFU model:** Introduces CPU-cap, spot-CPU, reserved-CPU, preemptible-mem -- decouples CPU and memory, adds urgency levels, allows lending idle resources.
- **Nearly-PFU billing:** Cost = reserved-CPU-time x Cr + borrowed-CPU-time x Cs - lent-CPU-time x Cs -- borrowers pay for spot resources, lenders get discounts.
- **Linux can't natively support CPU reservation:** CPU pinning gives exclusive access but blocks sharing; weighted sharing is sharing-friendly but gives incorrect reservations.
- **Leopard's kernel fix:** New cpu.resv_cpuset cgroup interface + modified CFS scheduler -- grants highest priority on reserved CPUs, best-effort sharing on spot CPUs.
- **Leopard results:** Provider throughput up 2.3x; customer cost down 34% (interactive) and 59% (batch). Billing models must be treated as core system design.

Chain Replication

- **Structure:** Nodes arranged head -> S1 -> S2 -> ... -> tail; writes enter at head, propagate forward; reads served only by tail.
- **Strong consistency:** Tail is the single commit point -- all reads see only committed state, no stale reads.
- **Pipelining:** Multiple writes flow through chain concurrently, distributing replication work across all nodes.
- **Write latency:** O(nL) where n = chain length, L = one-way link latency -- worse than primary/backup O(L) for writes.
- **Read latency:** O(2L) regardless of chain length -- faster than primary/backup which must wait for concurrent writes.
- **Idempotency:** Queries (reads) are idempotent; updates are not -- clients must track unacked ops and resend on timeout.
- **Head fails:** Master promotes next node to head, notifies clients; client resends any unacked update to new head.
- **Tail fails:** Master promotes predecessor to tail; new tail acks all previously seen-but-unacked updates to clients.
- **Inner node S fails:** Predecessor S- gets new successor S+; S- sends S+ only updates in its *sent list* (unacked), not full history.
- **Sent list:** Per-node list of updates forwarded but not yet acked by tail; pruned as acks propagate backward through chain.
- **Adding nodes:** New node T+ added at tail; T sends T+ its full state + in-flight updates; T+ becomes tail only when histories match.
- **Weak chain variant:** Allows reads from any node -- improves throughput for read-heavy workloads but sacrifices strong consistency (stale reads possible).
- **CRAQ extension:** Achieves strong consistency + read-any-node by checking sent list -- if non-empty (uncommitted updates pending), re-route to tail or stall until committed.